



DEPARTEMENT INFORMATIQUE

RAPPORT DE MINI PROJET

développement d'une application web de gestion d'absentéisme étudiant

Elaborée par

Rahma Chkel
Ines Landolsi

Supervisé par

Mr. Sofiene Ben Ahmed

Table des matières

Introduction Générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Présentation du Sujet	2
1.3 Problématique	2
1.4 Étude de l'existant	4
1.5 Solution proposée	6
1.6 Méthodologie de développement	7
1.7 Conclusion	7
2 Identification et spécification des besoins	8
2.1 Introduction	8
2.2 Identification des besoins	8
2.2.1 Identification des Acteurs	8
2.2.2 Besoins fonctionnels	9
2.2.3 Besoins non fonctionnels	9
2.3 Identification des cas d'utilisation	10
2.3.1 Diagramme de cas d'utilisation global	10
2.3.2 Diagrammes de cas d'utilisation raffinés	12
2.3.3 Conclusion	14
3 Analyse et conception	15
3.1 Introduction	15
3.2 Architecture logicielle de l'application	15
3.2.1 Description de l'architecture trois tiers	15
3.2.2 Architecture adoptée	16
3.3 Analyse et conception	17
3.3.1 Diagramme des classes	17
3.3.2 Diagrammes de séquence	19
3.4 Conclusion	24
4 Réalisation	25
4.1 Introduction	25
4.2 Environnement de développement	25
4.2.1 Environnement matériel	25
4.2.2 Environnement logiciel	25
4.3 Principales interfaces graphiques	27
4.3.1 Interface Tableau de bord Etudiant	27
4.3.2 Interface Justificatifs étudiant	28

TABLE DES MATIÈRES

4.3.3	Tableau de bord de l'enseignant	29
4.3.4	Interface "Faire l'appel"	30
4.3.5	Interface de gestion des justificatifs	31
4.3.6	Tableau de bord de l'agent administratif	32
4.3.7	Interface des statistiques globales	33
4.3.8	Interface de gestion des éliminations	35
4.4	Conclusion	35
Conclusion générale		36
Webographie		37

Table des figures

2.1	Diagramme de cas d'utilisation global.	11
2.2	Diagramme de cas d'utilisation de l'étudiant.	12
2.3	Diagramme de cas d'utilisation de l'enseignant.	13
2.4	Diagramme de cas d'utilisation de l'agent administratif.	14
3.1	Schéma de l'architecture trois tiers.	16
3.2	Diagramme des classes de l'application.	18
3.3	Diagramme de séquence "Enregistrer les présences".	19
3.4	Diagramme de séquence "Valider/Refuser un justificatif".	20
3.5	Diagramme de séquence "Déposer un justificatif".	21
3.6	Diagramme de séquence "Marquer un étudiant éliminé".	23
3.7	Diagramme de séquence "Notifier un étudiant éliminé".	24
4.1	Logo de VS Code.	26
4.2	Logo de React.	26
4.3	Logo de Django.	26
4.4	Logo de MongoDB.	27
4.5	Interface Tableau de bord étudiant.	28
4.6	Interface Justificatifs étudiant.	29
4.7	Tableau de bord de l'enseignant.	30
4.8	Interface de prise d'appel.	31
4.9	Interface de gestion des justificatifs.	32
4.10	Tableau de bord de l'agent administratif.	33
4.11	Interface des statistiques globales.	34
4.12	Interface de gestion des éliminations.	35

Liste des tableaux

1.1	Fonctionnalités de l'application Educenet	4
1.2	Fonctionnalités de l'application EduMS	5
1.3	Fonctionnalités de l'application EduPage	6
1.4	Planification des sprints pour le développement de la solution.	7

Introduction Générale

L'enseignement supérieur connaît aujourd'hui une transformation profonde marquée par la digitalisation de ses processus et services. Dans ce contexte, les établissements universitaires sont amenés à repenser leurs méthodes de gestion académique pour répondre aux défis contemporains. Parmi ces enjeux, l'absentéisme étudiant occupe une place centrale et constitue un facteur déterminant de l'échec académique. Les études montrent une corrélation forte entre le taux d'assiduité et la réussite aux examens, faisant de la gestion de l'absentéisme un levier essentiel pour améliorer les performances académiques. Pourtant, la plupart des établissements continuent d'utiliser des méthodes traditionnelles de suivi des présences qui ne permettent ni une détection précoce des situations à risque, ni une intervention rapide auprès des étudiants concernés.

C'est dans ce contexte que s'inscrit notre projet, réalisé dans le cadre de notre formation à l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse (ISSATSO). Notre travail porte sur la conception et le développement d'une application web dédiée à la gestion de l'absentéisme au sein des établissements d'enseignement supérieur. L'objectif principal est de proposer une solution moderne, centralisée et efficace qui permette de suivre en temps réel l'assiduité des étudiants, d'identifier rapidement les cas problématiques, et de mettre en place des actions préventives pour favoriser la réussite académique. L'application permettra aux enseignants de saisir rapidement les présences, aux étudiants de consulter leur taux d'assiduité, aux chefs de département d'obtenir des statistiques détaillées, et à l'administration d'avoir une vision globale pour prendre des décisions éclairées.

Le présent rapport retrace l'ensemble de notre démarche et est organisé en quatre chapitres principaux. Le premier chapitre présente le contexte général du projet, la problématique de l'absentéisme et la solution proposée. Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins, avec l'identification des acteurs et la modélisation des cas d'utilisation. Le troisième chapitre détaille la phase de conception, incluant l'architecture du système et les diagrammes UML. Enfin, le quatrième chapitre présente la réalisation concrète, l'environnement de développement, les choix technologiques et les principales fonctionnalités implémentées. À travers ce projet, nous espérons démontrer qu'une approche méthodologique rigoureuse peut apporter des solutions efficaces aux défis de la gestion universitaire tout en contribuant à l'amélioration de la réussite académique.

Chapitre 1

Contexte général du projet

1.1 Introduction

Dans le cadre de notre formation à l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse (ISSATSO), nous sommes amenés à réaliser un mini-projet portant sur l'étude, la conception et le développement d'une application de gestion du cursus universitaire. Ce projet s'inscrit dans une démarche de modernisation des processus académiques et vise à automatiser la gestion du parcours étudiant depuis l'inscription jusqu'à l'obtention du diplôme. Plus spécifiquement, notre projet se concentre sur la problématique cruciale de la gestion de l'absentéisme, un facteur déterminant dans la réussite académique des étudiants. L'objectif de ce chapitre est de présenter le contexte général du projet, d'identifier la problématique spécifique de l'absentéisme à laquelle il répond, d'analyser l'existant, et de proposer une solution adaptée aux besoins identifiés. Cette approche méthodologique nous permettra de définir un cadre clair pour les phases ultérieures de conception et de réalisation.

1.2 Présentation du Sujet

Dans le cadre de notre formation à l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse (ISSATSO), nous sommes amenés à réaliser un mini-projet portant sur l'étude, la conception et le développement d'une application de gestion du cursus universitaire. Ce projet s'inscrit dans une démarche de modernisation des processus académiques et vise à automatiser la gestion du parcours étudiant depuis l'inscription jusqu'à l'obtention du diplôme. Plus spécifiquement, notre projet se concentre sur la problématique cruciale de la gestion de l'absentéisme, un facteur déterminant dans la réussite académique des étudiants. L'objectif de ce chapitre est de présenter le contexte général du projet, d'identifier la problématique spécifique de l'absentéisme à laquelle il répond, d'analyser l'existant, et de proposer une solution adaptée aux besoins identifiés. Cette approche méthodologique nous permettra de définir un cadre clair pour les phases ultérieures de conception et de réalisation.

1.3 Problématique

L'absentéisme étudiant représente un défi majeur pour les établissements d'enseignement supérieur. Ce phénomène a des répercussions directes sur la réussite académique des étudiants. Il affecte également la qualité globale de l'enseignement.

Les méthodes traditionnelles de gestion de l'absentéisme montrent aujourd'hui leurs limites.

Ces approches reposent souvent sur des feuilles d'émargement papier ou des fichiers Excel dispersés. Elles présentent de nombreuses lacunes critiques. Tout d'abord, il est impossible de suivre en temps réel le taux d'absentéisme de chaque étudiant. Les enseignants éprouvent des difficultés à identifier rapidement les étudiants à risque qui nécessitent un accompagnement. En outre, l'absence de centralisation des données rend les statistiques peu fiables. Le processus de saisie reste par ailleurs chronophage pour les enseignants. Ces derniers doivent compiler manuellement les présences, ce qui augmente le risque d'erreurs. Les fraudes sont également possibles, comme les signatures par procuration ou les émargements multiples. La communication avec les étudiants concernant leur assiduité s'avère insuffisante. L'administration manque quant à elle d'une vision globale pour prendre des décisions basées sur des données fiables. Cette situation entraîne de multiples conséquences négatives. Le décrochage scolaire n'est pas détecté à temps. Des échecs académiques surviennent alors qu'une intervention précoce aurait pu les éviter. Le personnel subit une surcharge administrative importante. Enfin, les étudiants ne peuvent pas suivre leur propre assiduité. Face à ce constat, il devient impératif de mettre en place une solution informatique centralisée. Un outil moderne et efficace permettrait un suivi rigoureux de l'absentéisme. Il faciliterait également la mise en œuvre d'actions préventives.

1.4 Étude de l'existant

L'étude de l'existant est une étape cruciale pour étudier l'efficacité opérationnelle des solutions disponibles sur le marché. Cette analyse approfondie permet d'identifier les défis et les lacunes existants que notre projet doit surmonter pour atteindre le succès.

TABLE 1.1 – Fonctionnalités de l'application Educanet

Nom de l'application	Educant
Logo de l'application	
Fondée en	2017
Société	STE Educant Tunisie
Pays	Sousse, Tunisie
Services offerts	- Suivi des absences, retards et éliminations en temps réel. - Envoi de notifications et circulaires par SMS - Import/export de données Excel
Critique de l'application	- Module absences non spécialisé : intégré à un ERP généraliste, sans analytique avancée ni rapports dédiés. - Absence d'outils prédictifs poussés. - Adoption quasi nulle dans les universités publiques tunisiennes.
Accès à l'application	https ://educant.pro/

TABLE 1.2 – Fonctionnalités de l'application EduMS

Nom de l'application	EduMS
Logo de l'application	
Fondée en	2020
Société	DEVNET
Pays	Tunis, Tunisie
Services offerts	- Suivi des absences et retards en temps réel. - Génération automatique de statistiques et rapports pluriannuels. - Vision graphique fonctionnelle pour aide à la décision.
Critique de l'application	- Solution généraliste (ERP global), le module absences n'est pas une solution dédiée spécialisée - Absence d'analyse prédictive avancée ou d'IA pour détecter les profils d'étudiants à risque.
Accès à l'application	https://edums.tn/

TABLE 1.3 – Fonctionnalités de l'application EduPage

Nom de l'application	EduPage
Logo de l'application	
Fondée en	2012
Société	aSc Applied Software Consultants.
Pays	Slovaquie
Services offerts	- Enregistrement des absences par les enseignants - Justification des absences en ligne par les étudiants/parents. - Notifications automatiques aux parents et administration. - Statistiques et rapports d'assiduité. - Détection des zones problématiques
Critique de l'application	- Solution internationale non conçue spécifiquement pour le système universitaire tunisien (LMD) - Non adaptée aux spécificités réglementaires du Ministère de l'Enseignement Supérieur tunisien. - Moins adapté aux très grandes universités publiques à fort volume d'étudiants (10 000+)
Accès à l'application	https://edums.tn/

1.5 Solution proposée

Pour répondre à la problématique de la gestion de l'absentéisme, nous proposons le développement d'une application web moderne, centralisée et spécialisée dans le suivi des présences et absences. Cette solution sera conçue selon une architecture modulaire permettant une évolutivité et une maintenance facilitées. L'application offrira une interface intuitive et responsive, accessible depuis n'importe quel appareil connecté (ordinateur, tablette, smartphone), permettant aux enseignants de saisir rapidement les présences lors de chaque séance. Le système intégrera des fonctionnalités essentielles et innovantes : enregistrement rapide et simplifié des présences par séance avec possibilité de marquage collectif ou individuel, calcul automatique en temps réel du taux d'absentéisme par étudiant et par matière, génération automatique d'alertes lorsqu'un étudiant dépasse le seuil d'absence autorisé, tableaux de bord interactifs pour visualiser les statistiques d'absentéisme à différents niveaux (étudiant, cours, département), système de notifications automatiques envoyées aux étudiants concernant leur situation d'assiduité, génération de rapports détaillés et personnalisables pour l'administration, historique complet et traçable de toutes les présences et absences, et possibilité pour les étudiants de consulter leur propre assiduité et de justifier leurs absences.

1.6 Méthodologie de développement

Pour le développement de ce module de gestion de l'absentéisme, nous avons opté pour une méthodologie Agile Scrum, adaptée aux projets académiques de courte durée. Cette approche itérative et incrémentale nous permet de :

- Livrer des fonctionnalités opérationnelles à chaque sprint
- S'adapter aux retours et ajustements en cours de développement
- Maintenir une visibilité constante sur l'avancement du projet
- Prioriser les fonctionnalités selon leur valeur métier

TABLE 1.4 – Planification des sprints pour le développement de la solution.

Sprint	Objectif
Sprint 1	Authentification et architecture de base (inscription, connexion, gestion des rôles)
Sprint 2	Gestion des utilisateurs, départements, filières, groupes et matières
Sprint 3	Module de prise d'appel en temps réel (présent, absent, retard)
Sprint 4	Gestion des justificatifs (dépôt, validation, refus), gestion des éliminations et notifications
Sprint 5	Tableaux de bord (étudiant, enseignant, administrateur) et statistiques
Sprint 6	Tests finaux, optimisation des performances, correction des bugs et documentation

1.7 Conclusion

Ce premier chapitre a permis de poser les fondations de notre projet en présentant son contexte, ses enjeux et ses objectifs. Nous avons mis en évidence la problématique critique que représente l'absentéisme dans l'enseignement supérieur et les limitations importantes des méthodes traditionnelles de suivi des présences. L'analyse de l'existant a révélé l'absence de solutions adaptées et efficaces, justifiant pleinement le développement d'une application sur mesure répondant aux besoins spécifiques identifiés. La solution proposée vise à offrir un système moderne, et efficace qui permettra un suivi rigoureux de l'assiduité. Dans le chapitre suivant, nous procéderons à une analyse détaillée des besoins fonctionnels et non fonctionnels et nous identifierons précisément les différents acteurs de l'application.

Chapitre 2

Identification et spécification des besoins

2.1 Introduction

Ce deuxième chapitre est dédié à la spécification des besoins de notre application. Après avoir posé le cadre général du projet, nous identifions ici les différents acteurs qui interagiront avec le système, ainsi que les fonctionnalités qui leur sont destinées (besoins fonctionnels). Nous définissons également les contraintes techniques et qualitatives (besoins non fonctionnels) que la solution devra respecter. Enfin, nous modélisons ces interactions à travers des diagrammes de cas d'utilisation, offrant ainsi une vision claire du périmètre fonctionnel de l'application avant d'entamer la phase de conception.

2.2 Identification des besoins

2.2.1 Identification des Acteurs

Les acteurs de notre projet représentent tous les utilisateurs et systèmes externes qui interagiront directement avec l'application. Nous les classons en deux catégories : les acteurs primaires, qui initient les interactions pour utiliser les fonctionnalités principales, et les acteurs secondaires, qui fournissent des services ou des informations complémentaires.

Acteurs

Agent administratif : Responsable de la supervision et de la gestion globale de l'absentéisme au sein de l'établissement universitaire. Il dispose d'une vision transversale sur l'assiduité des étudiants à tous les niveaux (département, groupe, matière) et assure le suivi administratif des situations critiques.

Enseignant : Acteur central du système, chargé de l'enregistrement quotidien des présences et absences lors de ses séances d'enseignement. Il assure le suivi pédagogique de l'assiduité de ses groupes d'étudiants.

Étudiant : Utilisateur consultatif qui accède à ses données personnelles d'assiduité. Il peut suivre son taux d'absentéisme, justifier ses absences et recevoir des alertes concernant sa situation vis-à-vis du seuil réglementaire.

2.2.2 Besoins fonctionnels

L'analyse des besoins fonctionnels est une étape essentielle pour déterminer les fonctionnalités que notre application doit accomplir.

Agent administratif :

- ★ Consulter un tableau de bord global de l'absentéisme de l'établissement
- ★ Visualiser les statistiques d'absentéisme par département et par groupe
- ★ Identifier les étudiants en situation critique (dépassement de seuil)
- ★ Export de rapports PDF sur l'assiduité pour des périodes définies.
- ★ Gestion des Alertes
- ★ Gestion de l'Élimination

Enseignant :

- ★ Visualiser la liste des étudiants du groupe
- ★ Marquer tous les étudiants comme présents/absents (absence collectif)
- ★ Marquer individuellement les étudiants absents
- ★ Modifier une présence déjà enregistrée
- ★ Visualisation des justificatifs déposés par les étudiants pour ses cours
- ★ Validation ou rejet de ces justificatifs
- ★ Visualiser les statistiques par groupe

Etudiant :

- ★ Consultation de la liste de ses absences
- ★ Dépôt d'un justificatif (format PDF/Image) pour une absence spécifique
- ★ Suivi de l'état de sa demande (en attente, validée, refusée)
- ★ Réception d'alertes en cas de dépassement d'un seuil d'absence

2.2.3 Besoins non fonctionnels

Les besoins non fonctionnels sont les caractéristiques et contraintes qui affectent les performances, la sécurité et d'autres éléments essentiels d'un système.

- **Performance :** L'application doit être capable de gérer un grand nombre d'utilisateurs et de données dans des délais raisonnables.
- **Sécurité :** Les données doivent être cryptées lors de leur transmission et stockées de manière sécurisée, en suivant les meilleures pratiques en matière de sécurité informatique pour protéger la confidentialité des informations personnelles des utilisateurs.
- **Disponibilité :** L'application doit être opérationnelle à tout moment, 24 heures sur 24 et 7 jours sur 7, afin de répondre aux besoins des utilisateurs sans interruption.
- **Fiabilité :** L'application doit fonctionner de manière stable et sans erreur, minimisant les failles et les risques de panne.
- **Ergonomie :** Les interfaces utilisateur doivent être conçues de manière intuitive et conviviale, facilitant la navigation et l'interaction avec l'application.

2.3 Identification des cas d'utilisation

Les diagrammes de cas d'utilisation mettent en lumière les interactions entre les acteurs et l'application. Pour visualiser ces interactions, nous avons utilisé l'UML, qui est destiné à faciliter la conception des documents nécessaires au développement d'un logiciel orienté objet, comme standard de modélisation de l'architecture logicielle [1]. Les diagrammes de cas d'utilisation permettent de représenter le comportement fonctionnel d'un système logiciel. Ils se focalisent sur les interactions entre les utilisateurs et le système, en décrivant les différentes fonctionnalités offertes et comment les utilisateurs y accèdent.

2.3.1 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global, présenté dans la figure 2.1, présente l'ensemble des fonctionnalités offertes par la plateforme de gestion d'absentéisme, ainsi que les échanges entre les divers acteurs impliqués. Il constitue un moyen efficace de communiquer et de saisir les exigences des utilisateurs, offrant une représentation claire des diverses fonctionnalités du système et de leur utilisation.

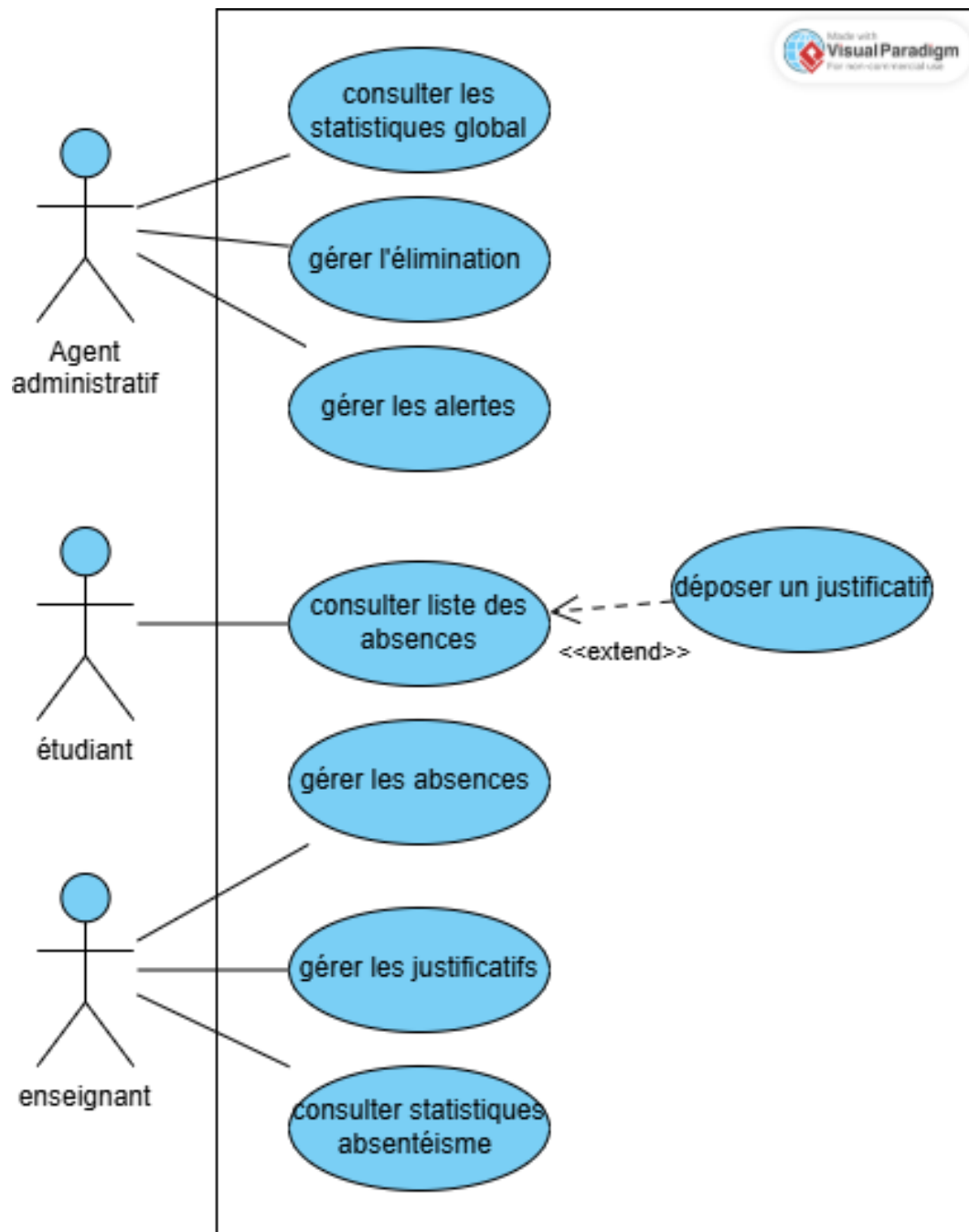


FIGURE 2.1 – Diagramme de cas d'utilisation global.

2.3.2 Diagrammes de cas d'utilisation raffinés

Diagramme de cas d'utilisation de l'étudiant

L'étudiant peut consulter son relevé des d'absences, déposer un justificatif et suivre l'état de sa demande. La figure 2.2 illustre ces différentes interactions.

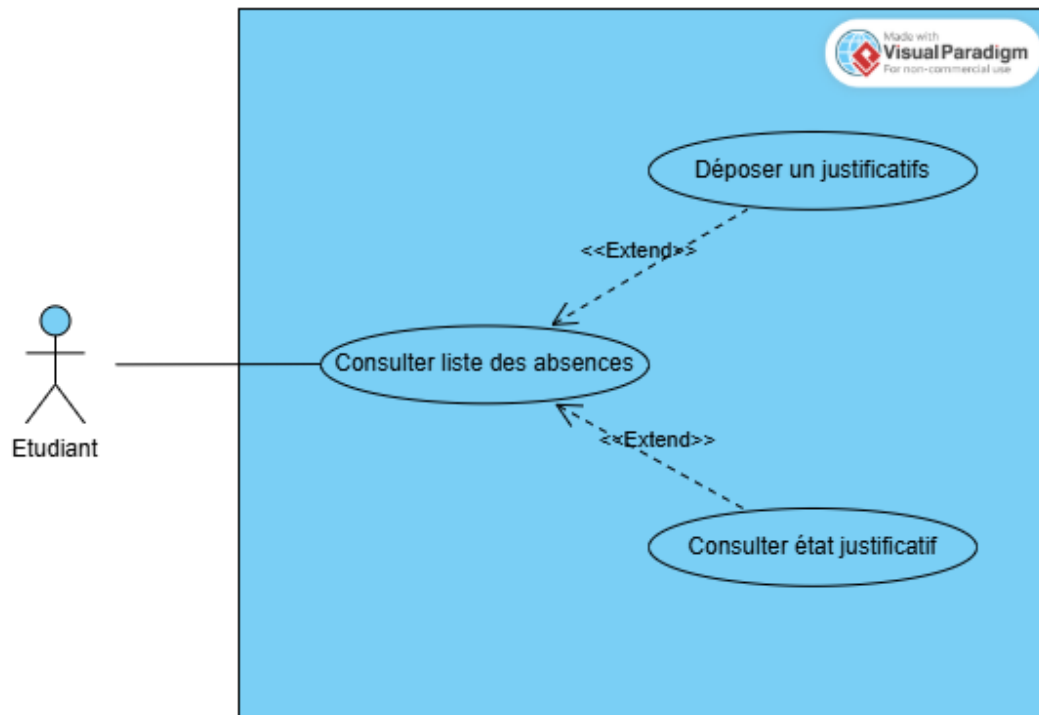


FIGURE 2.2 – Diagramme de cas d'utilisation de l'étudiant.

Diagramme de cas d'utilisation de l'enseignant

L'enseignant, après authentification, peut enregistrer les présences des étudiants (individuellement ou en masse), modifier une présence déjà enregistrée, consulter la liste des étudiants, visualiser les justificatifs déposés, les valider ou les refuser avec un commentaire. Il peut également consulter les statistiques d'absentéisme par groupe et visualiser le taux d'absentéisme calculé automatiquement par le système. La figure 2.3 détaille ces fonctionnalités.

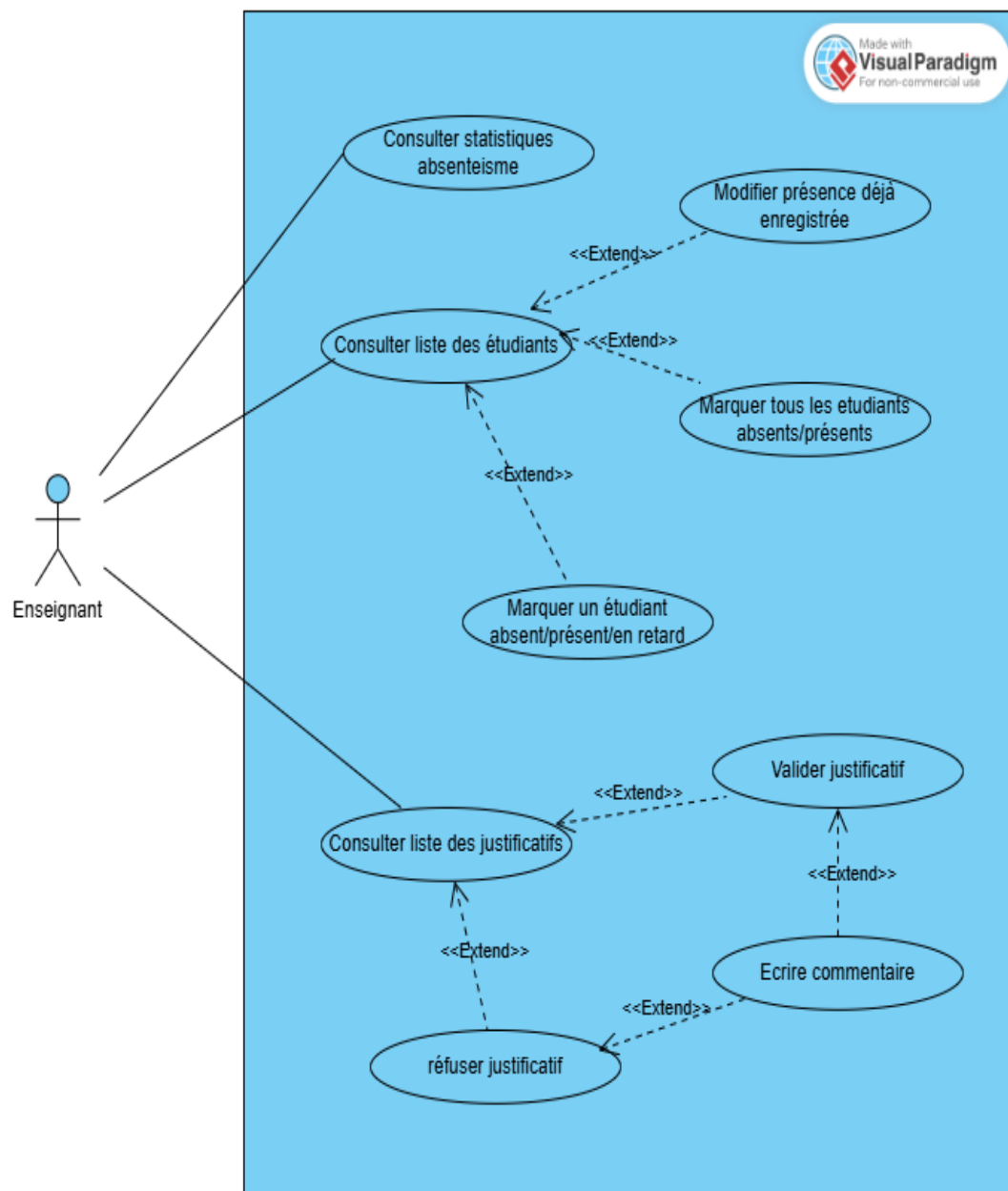


FIGURE 2.3 – Diagramme de cas d'utilisation de l'enseignant.

Diagramme de cas d'utilisation de l'agent administratif

L'agent administratif a accès à un tableau de bord global lui permettant de visualiser les statistiques d'absentéisme. Il consulte la liste des étudiants en situation critique, marque un étudiant comme "éliminé" d'une matière, et envoyer des alertes automatiques. La figure 2.4 présente ces cas d'utilisation.

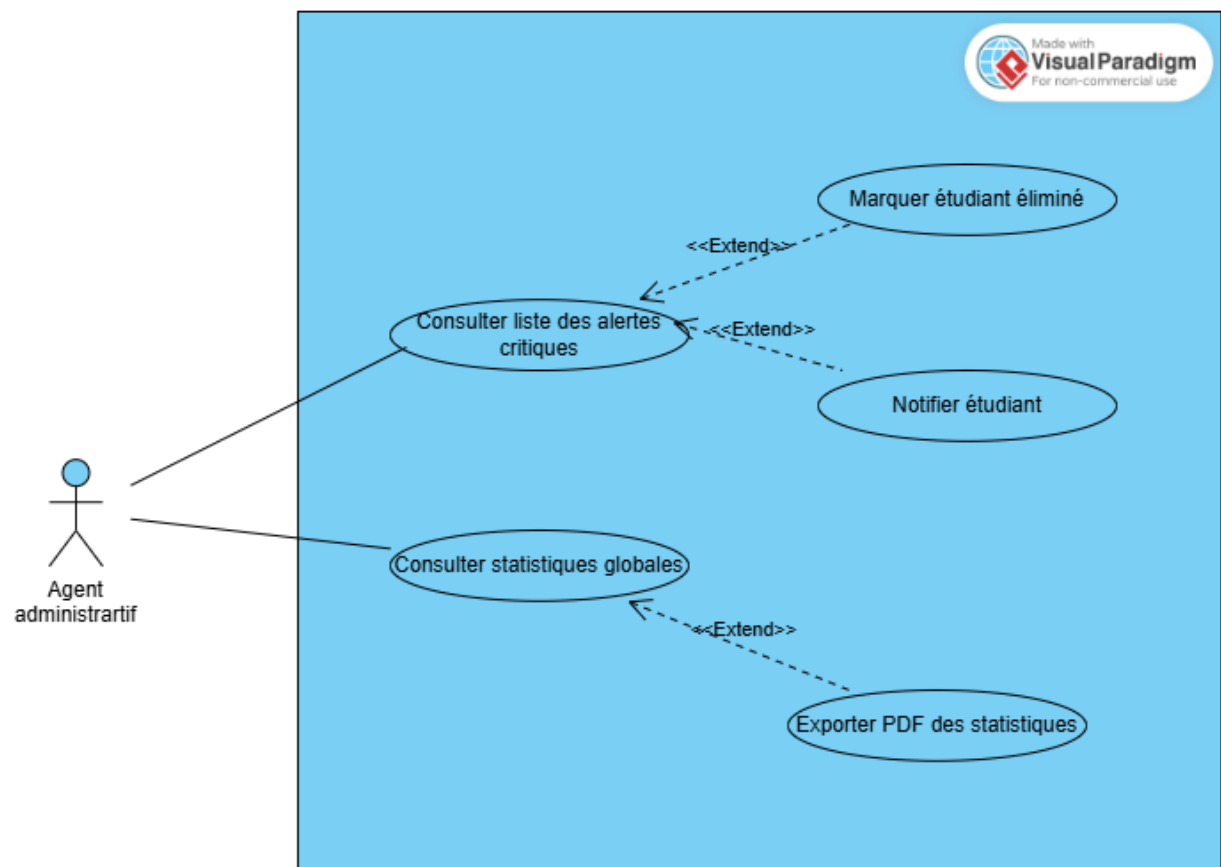


FIGURE 2.4 – Diagramme de cas d'utilisation de l'agent administratif.

2.3.3 Conclusion

Nous avons étudié les exigences fonctionnelles et non fonctionnelles de notre application de gestion d'absentéisme, en identifiant les principaux acteurs que sont l'étudiant, l'enseignant et l'agent administratif. Les diagrammes de cas d'utilisation élaborés mettent en lumière les interactions entre ces acteurs et l'application.

De plus, nous avons réalisé une analyse approfondie des exigences de performance et de scalabilité de notre projet afin de garantir la création et la mise en place d'une application de gestion d'absentéisme efficace et robuste, répondant aux besoins des établissements universitaires.

Chapitre 3

Analyse et conception

3.1 Introduction

Après l'analyse et la spécification des besoins du projet, nous passons, dans ce chapitre, à la préparation de la phase de réalisation, où une conception détaillée de l'application est proposée. Cette étape implique spécifiquement la définition de l'architecture de l'application.

3.2 Architecture logicielle de l'application

En informatique l'architecture logicielle décrit d'une manière symbolique et schématique les différents éléments d'un ou de plusieurs systèmes informatiques, leurs interrelations et leurs interactions [2]. Cette architecture fournit un cadre indispensable qui guide la prise de décisions stratégiques concernant la répartition des responsabilités, la communication entre les différents composants, ainsi que la gestion des données et d'autres aspects essentiels. Dans le contexte de notre projet, notre application est composée de services frontend et backend. Le développement front-end se concentre sur la création d'interfaces conviviales facilitant l'interaction entre l'utilisateur et l'application, tandis que le développement back-end assure l'accès aux données, aux services et à d'autres systèmes sur lesquels notre application s'appuie.

3.2.1 Description de l'architecture trois tiers

établie qui organise les applications en trois niveaux informatiques logiques et physiques : le niveau de présentation, ou interface utilisateur ; le niveau d'application, où les données sont traitées ; et le niveau d'accès aux données, où les données d'application sont stockées et gérées.

Le principal avantage de l'architecture à trois niveaux est que chaque niveau fonctionne sur sa propre infrastructure et que les trois peuvent donc être développés simultanément par des équipes de développement différentes. Et il peut être mis à jour ou dimensionné selon les besoins, sans impacter les autres niveaux. [3].

- **Couche de présentation (frontend) :** Le modèle dans l'architecture MVC représente la structure des données de l'application ainsi que les règles métier associées. Il est chargé de la gestion de la logique métier, comprenant la manipulation des données, les validations et les interactions avec la base de données. De plus, il fonctionne de manière indépendante de l'interface utilisateur et des interactions avec l'utilisateur.
- **Couche métier (backend / application) :** Elle implémente la logique fonctionnelle de l'application (règles de gestion, calculs, validations, workflows). Elle reçoit les requêtes de la couche présentation, les traite en interagissant si nécessaire avec la couche données, et

renvoie les réponses appropriées. Cette couche est indépendante de l'interface utilisateur et du système de stockage.

- **Couche de données (base de données) :** Elle assure le stockage persistant et la gestion des données. Elle est généralement constituée d'un système de gestion de base de données (relationnelle ou NoSQL). Seule la couche métier est autorisée à interagir avec elle, ce qui renforce la sécurité et la cohérence.

Les communications entre couches se font via des protocoles standards (HTTP, TCP/IP, appels de procédure distants). Les flux sont typiquement unidirectionnels : présentation → métier → données, puis retour des données vers la présentation.

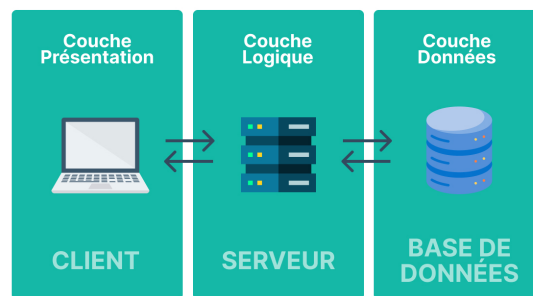


FIGURE 3.1 – Schéma de l'architecture trois tiers.

3.2.2 Architecture adoptée

Dans notre projet, nous avons choisi une architecture en trois couches distinctes garantissant une séparation claire des responsabilités entre les différentes parties de l'application, ce qui favorise une meilleure maintenabilité et extensibilité du code. Notre architecture se compose ainsi de trois composants principaux :

- **Le modèle :** Responsable de la gestion des données et des opérations liées à la base de données MongoDB. Des classes représentant les entités de l'application ont été créées (Utilisateur, Enseignant, Etudiant, AgentAdministratif, Matiere, Groupe, Seance, Presence, Justificatif), correspondant aux collections de la base de données. Nous utilisons **MongoEngine** comme ODM (Object Document Mapper) pour faciliter l'interaction avec la base de données depuis notre application Django.
- **La vue :** Développée en utilisant **React** pour créer l'interface utilisateur. Des composants React ont été créés pour afficher les différentes vues (tableau de bord, prise d'appel, gestion des justificatifs, statistiques) et interagir avec l'utilisateur. Ils récupèrent les données du backend via des requêtes API REST et les affichent de manière appropriée, tout en envoyant des requêtes au backend pour effectuer des opérations sur les données.
- **Le contrôleur :** Implémenté avec **Django REST Framework**, les contrôleurs sont responsables de recevoir les requêtes HTTP provenant du frontend (React), d'interagir avec les composants du modèle (via MongoEngine) et de renvoyer les réponses appropriées au format JSON.

En résumé, l'architecture de notre application se compose des trois couches suivantes :

Frontend (React) ↔ Backend (Django REST Framework) ↔ Base de données (MongoDB)

Cette architecture permet une séparation nette entre la logique de présentation (React), la

logique métier (Django) et le stockage des données (MongoDB), facilitant ainsi le développement, les tests et la maintenance de l'application.

3.3 Analyse et conception

3.3.1 Diagramme des classes

Le diagramme des classes est un schéma utilisé en génie logiciel pour présenter les classes et les interfaces des systèmes ainsi que leurs méthodes et leurs relations. Il modélise la structure statique d'une application orientée objet, indépendamment des aspects temporels et dynamiques [4].

Le diagramme des classes est un outil incontournable dans la conception orientée objet. Il simplifie la compréhension et la communication au sein de l'équipe de développement en offrant une visualisation claire des entités principales et de leurs interactions.

Représentation graphique du diagramme des classes

Le diagramme des classes présenté dans la figure 3.2 illustre la structure statique de notre application de gestion d'absentéisme. On y retrouve les principales entités du système :

- **Utilisateur** : classe abstraite représentant tous les utilisateurs (attributs communs : nom, prénom, email, CIN, téléphone, mot de passe)
- **Etudiant** : hérite de **Utilisateur**, avec ajout du numéro d'inscription et l'appartenance à un groupe
- **Enseignant** : hérite de **Utilisateur**, avec ajout de la spécialité
- **AgentAdministratif** : hérite de **Utilisateur**, avec ajout du poste
- **Departement** : entité représentant les départements de l'institut
- **Filiere** : entité représentant les filières (ex : GL, INFO) liée à un département
- **Groupe** : entité représentant les groupes d'étudiants (ex : 2A-INFO) lié à une filière
- **Matiere** : entité représentant les matières enseignées (nom, code, volume horaire, seuil d'absence)
- **Seance** : entité représentant les séances de cours (date, horaire, type, matière, groupe, enseignant)
- **Presence** : entité représentant la présence d'un étudiant à une séance (statut : présent, absent, retard)
- **Justificatif** : entité représentant un justificatif déposé par un étudiant (fichier, statut, commentaire)
- **Elimination** : entité représentant l'élimination d'un étudiant pour absentéisme excessif
- **Alerte** : entité représentant les alertes générées automatiquement
- **Notification** : entité représentant les notifications envoyées aux utilisateurs

Les associations entre ces classes (héritage, composition, agrégation) ainsi que les multiplicités sont clairement représentées sur le diagramme, ce qui permet de comprendre les relations et les dépendances entre les différentes données manipulées par l'application.

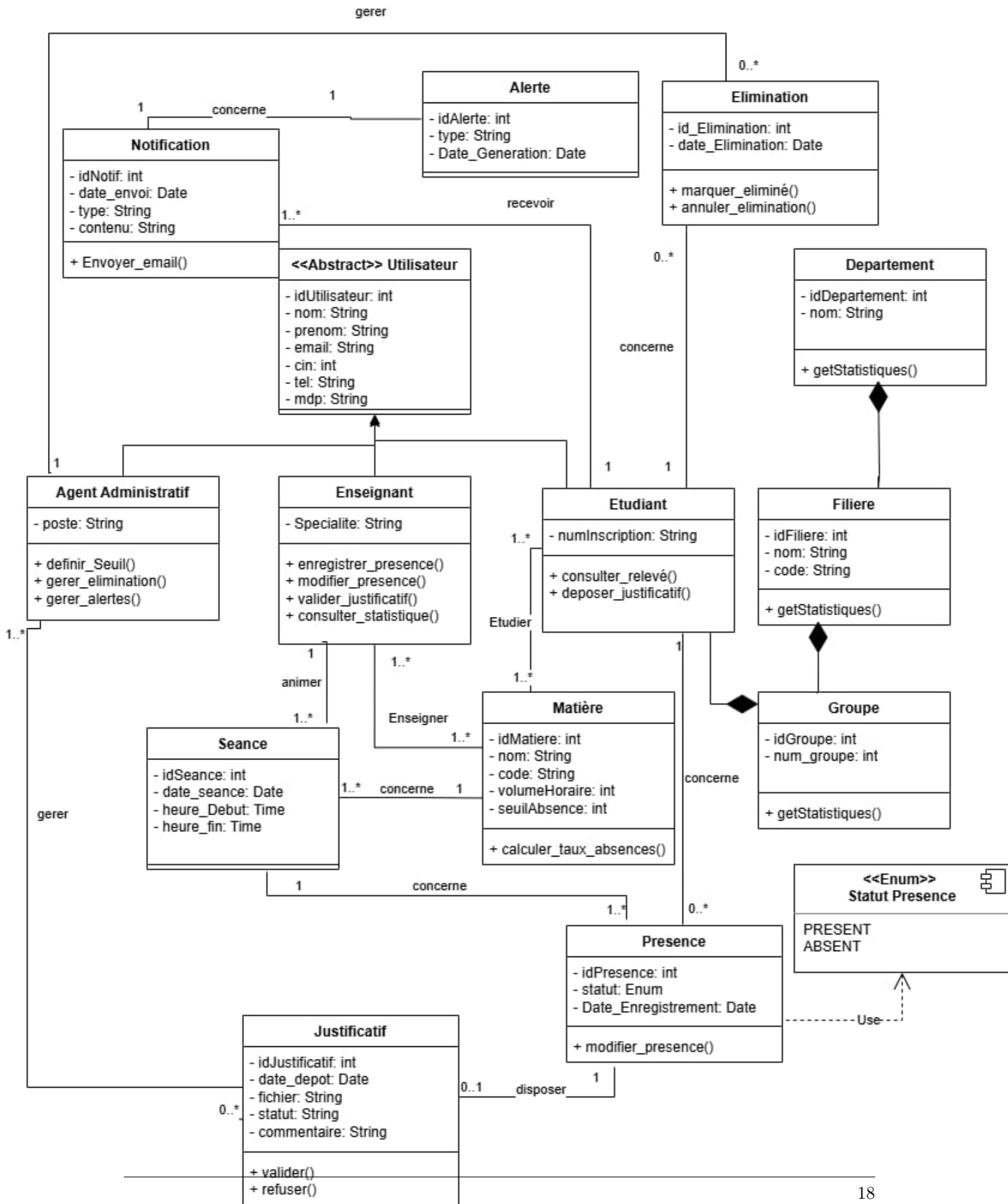


FIGURE 3.2 – Diagramme des classes de l'application.

3.3.2 Diagrammes de séquence

Les diagrammes de séquence sont la présentation graphique des interactions entre les acteurs et le système selon un ordre chronologique dans la modélisation UML [5]. Leur but principal est de visualiser l'enchaînement des actions dans le temps, ce qui facilite la compréhension du déroulement de l'exécution de l'application.

Diagramme de séquence "Enregistrer les présences"

Ce diagramme décrit le processus de saisie des présences par un enseignant. Après authentification, l'enseignant sélectionne une séance, ce qui déclenche le chargement de la liste des étudiants depuis la base de données. Il peut ensuite marquer un étudiant absent individuellement, ou marquer tous les étudiants absents collectivement. Dans les deux cas, les présences sont enregistrées dans la base de données.

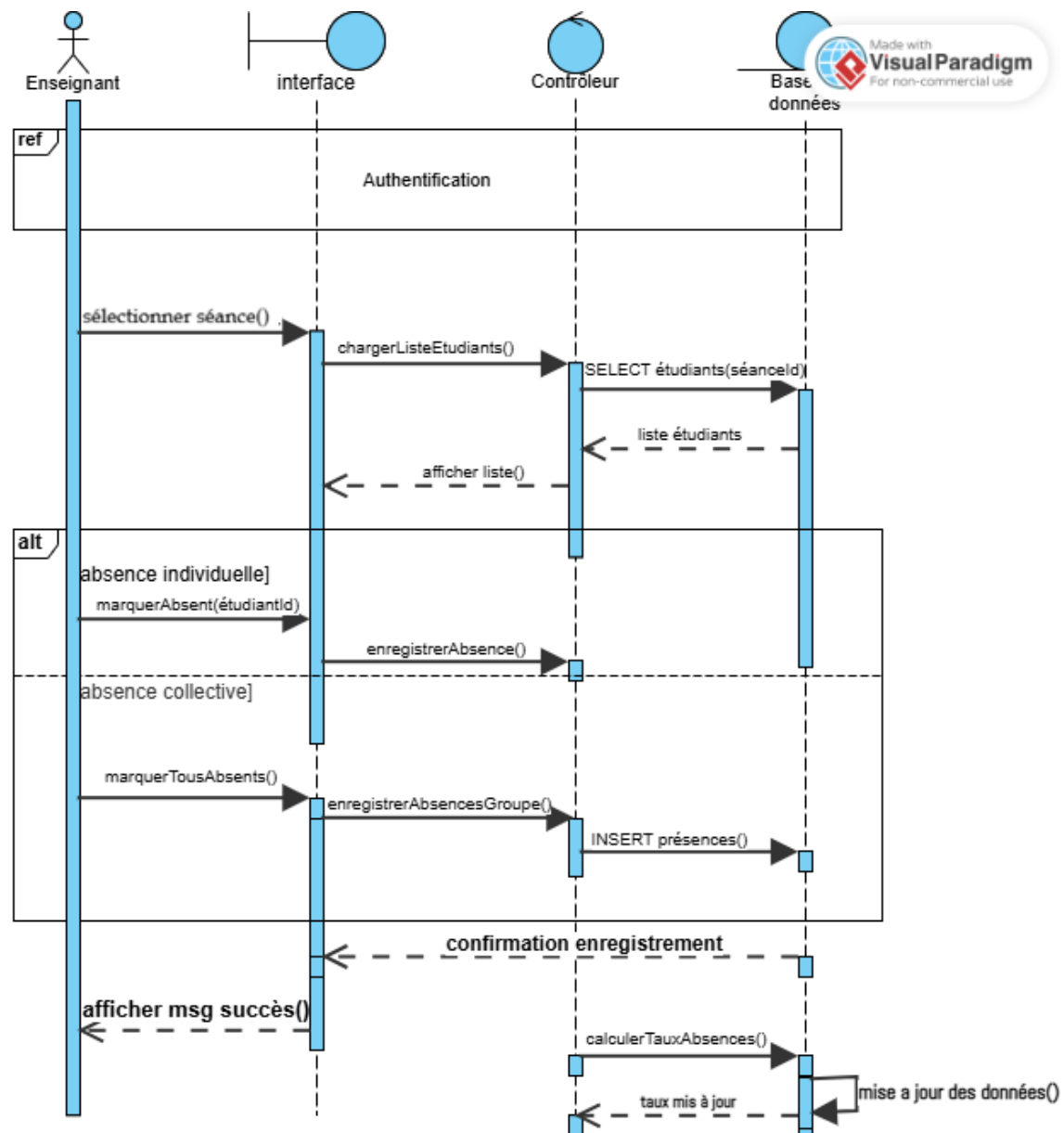


FIGURE 3.3 – Diagramme de séquence "Enregistrer les présences".

3.3.2.2 Diagramme de séquence «Valider/Refuser un justificatif»

Ce diagramme décrit le traitement des justificatifs d'absence par un enseignant. Après authentification, l'enseignant consulte la liste des justificatifs soumis, sélectionne un justificatif et choisit de le valider ou de le refuser. Dans le cas d'une validation, `validerJustificatif()` est appelé et le statut est mis à jour via `UPDATE` dans la base de données. Dans le cas d'un refus, `refuserJustificatif(justificatifId, commentaire)` est appelé. Dans les deux cas, une confirmation est retournée et un message de succès est affiché à l'enseignant.

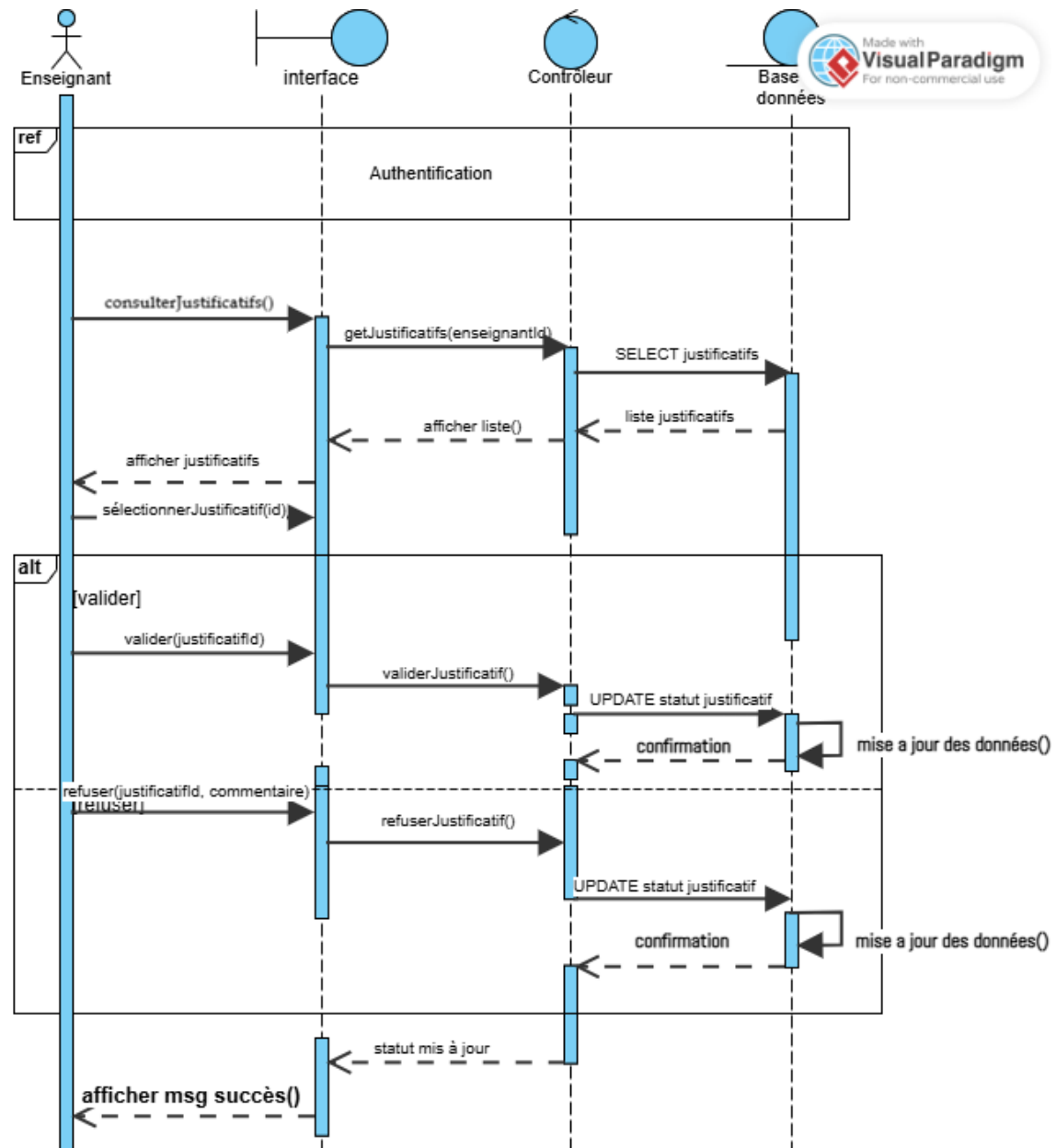


FIGURE 3.4 – Diagramme de séquence "Valider/Refuser un justificatif".

3.3.2.3 Diagramme de séquence «Déposer un justificatif»

Ce diagramme illustre le processus de soumission d'un justificatif d'absence par un étudiant. Après authentification, l'étudiant consulte ses absences et justificatifs via `consulterAbsencesETjustificatifs()`, ce qui déclenche une requête `SELECT` filtrée par son identifiant. La liste est affichée. L'étudiant dépose ensuite un fichier justificatif associé à une absence. Si le format est invalide, un message d'erreur est affiché. Si valide, le justificatif est soumis avec le statut *en attente* et inséré dans la base de données. Un message de succès confirme la soumission.

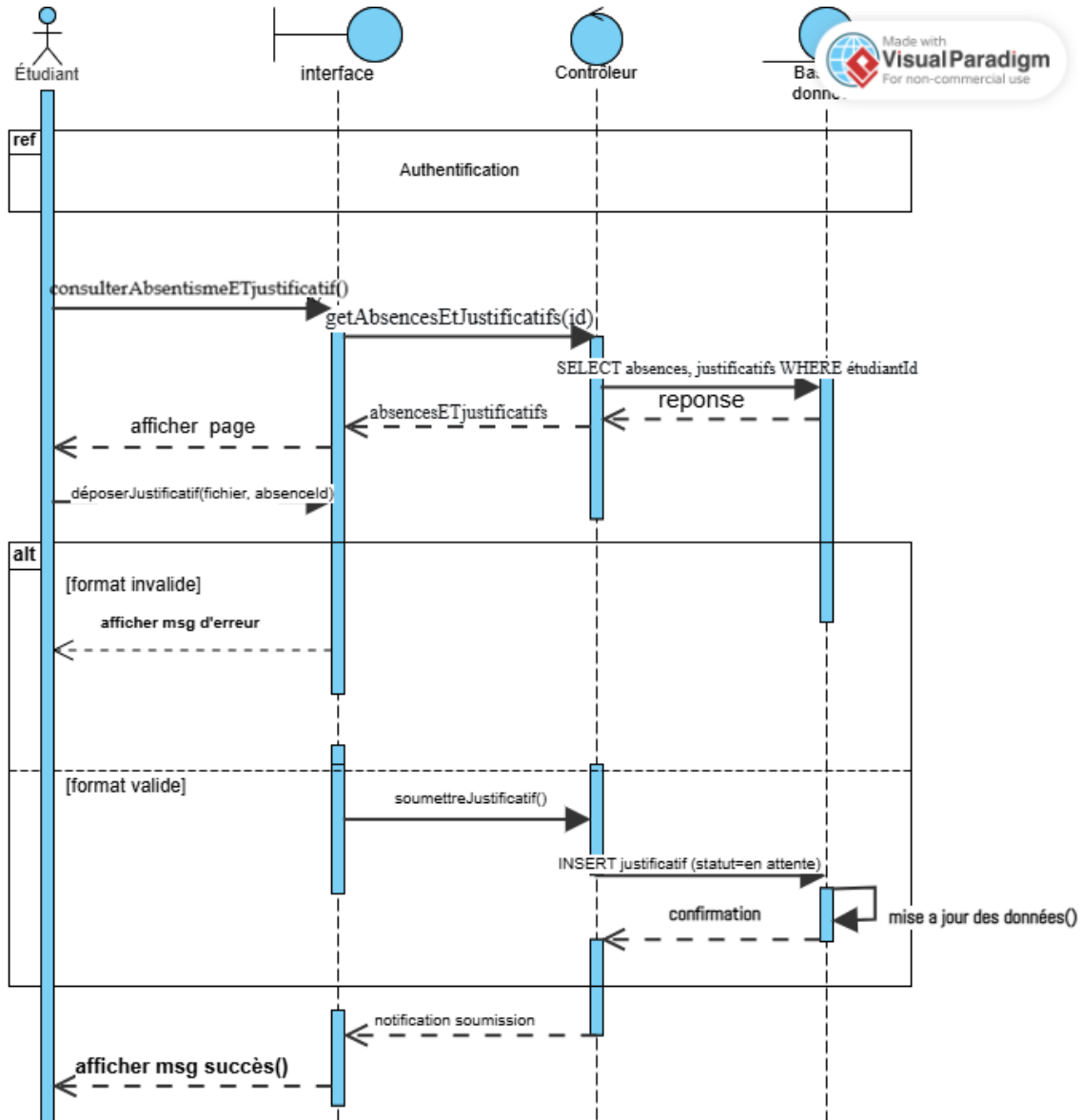


FIGURE 3.5 – Diagramme de séquence "Déposer un justificatif".

3.3.2.4 Diagramme de séquence «Marquer un étudiant éliminé»

Ce diagramme illustre la gestion des éliminations par l'agent administratif. Après authentification, l'agent accède à la liste des étudiants à risque, sélectionne un étudiant et clique sur

"Marquer éliminé". Une confirmation lui est demandée. Si confirmée, `marquerElimine(étudiant, matière)` est soumis au contrôleur, l'élimination est enregistrée dans la base de données et un message de confirmation s'affiche. Si annulée, la liste initiale des étudiants est restaurée.

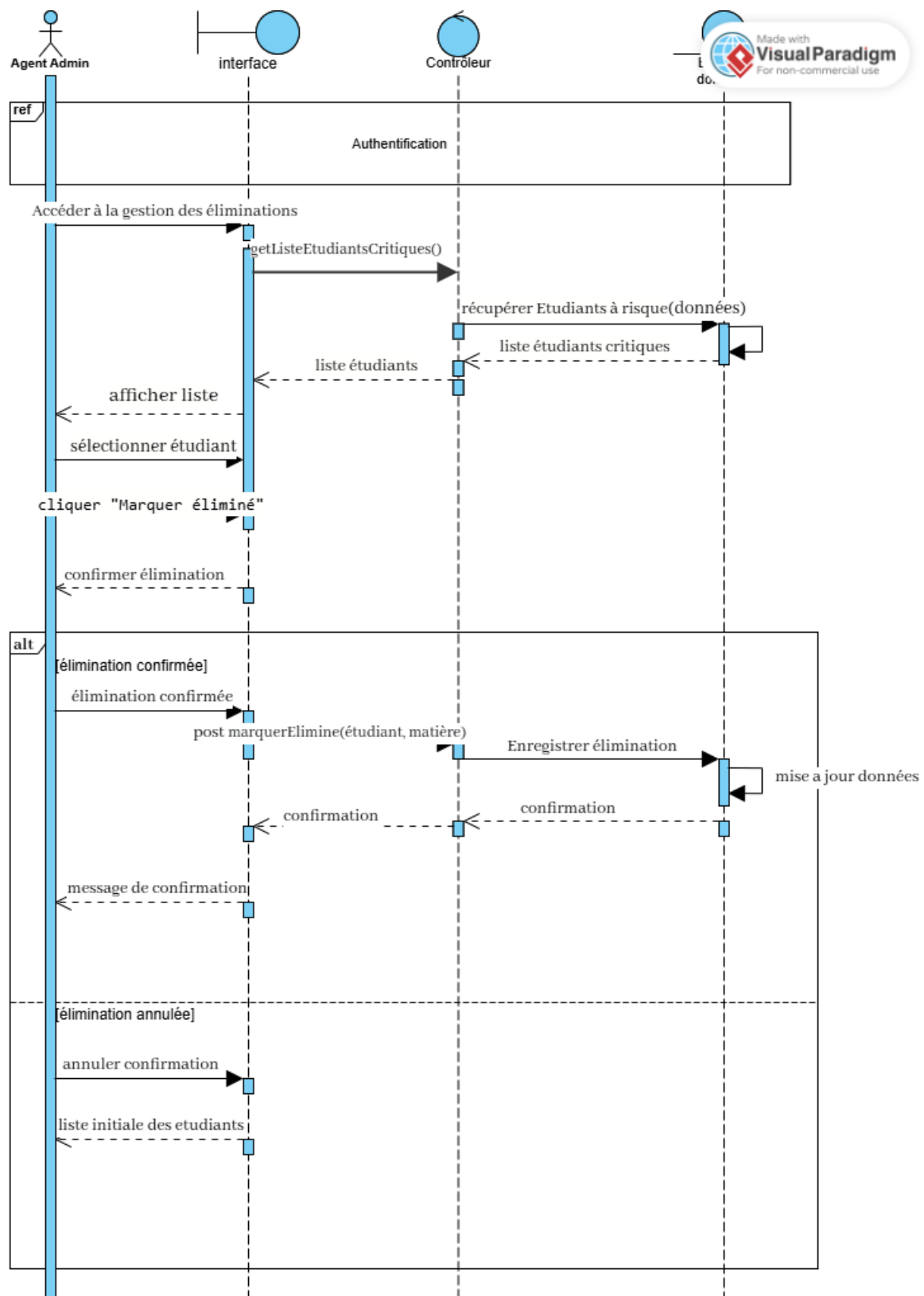


FIGURE 3.6 – Diagramme de séquence "Marquer un étudiant éliminé".

3.3.2.5 Diagramme de séquence «Notifier un étudiant éliminé»

Ce diagramme décrit la notification d'un étudiant éliminé par l'agent administratif. Après authentification, l'agent clique sur "Notifier étudiant" et confirme l'envoi. Une requête POST `/api/admin/eliminations/{elimination_id}/notifier/` est transmise au contrôleur, qui recherche l'élimination dans la base de données, génère et envoie un email à l'étudiant. Un message de succès est affiché à l'agent.

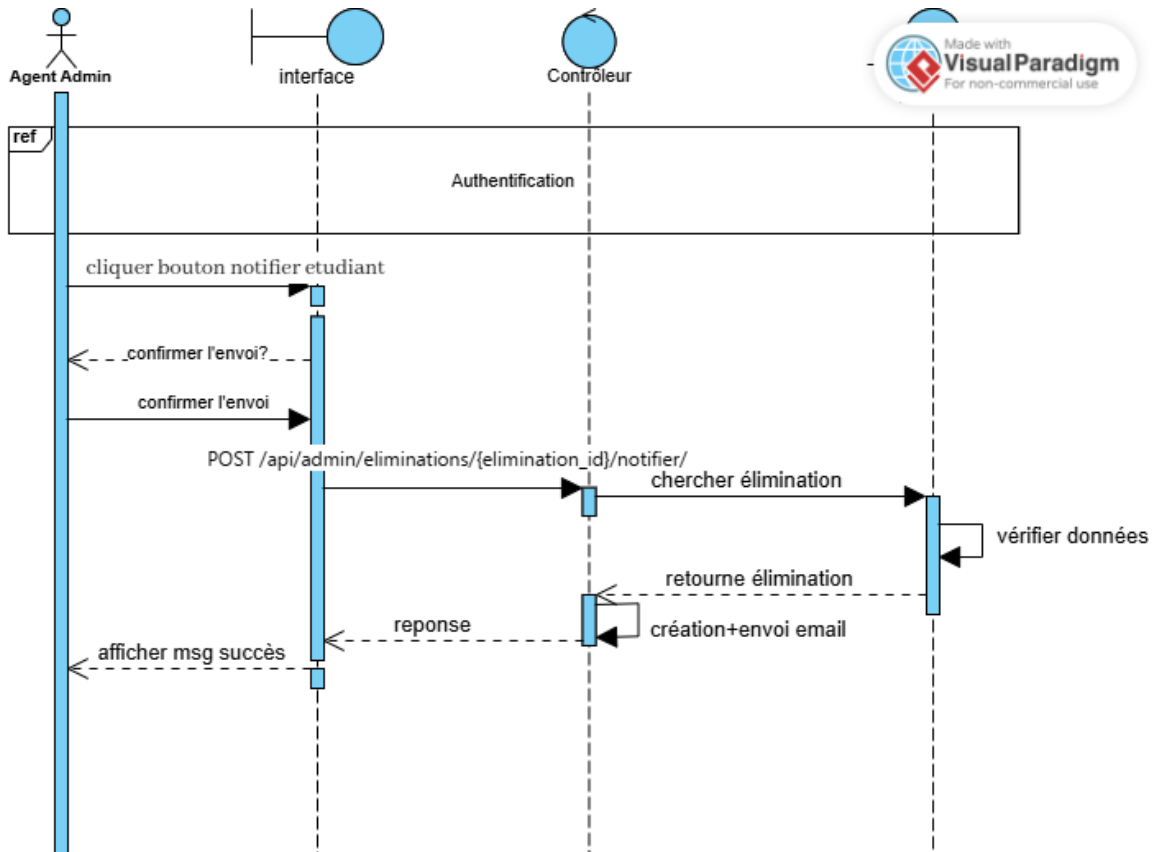


FIGURE 3.7 – Diagramme de séquence "Notifier un étudiant éliminé".

3.4 Conclusion

Ce chapitre présente une vue conceptuelle de la solution à mettre en place. Il expose les différents diagrammes UML pour mieux comprendre les fonctionnalités offertes et pour mieux représenter la communication entre les différents objets du projet. Le chapitre suivant présente la partie mise en œuvre de l'application.

Chapitre 4

Réalisation

4.1 Introduction

Après avoir achevé la phase de conception, nous entrons maintenant dans l'étape de l'implémentation. Tout aussi vitale que la phase précédente, l'implémentation consiste à concrétiser le modèle conceptuel en composants logiciels concrets, formant ainsi l'application finale. Ce chapitre clôture notre rapport en présentant l'environnement matériel et logiciel dans lequel nous avons travaillé, ainsi que les choix technologiques opérés pour mener à bien cette transformation

4.2 Environnement de développement

4.2.1 Environnement matériel

La programmation a été réalisée sur un ordinateur doté des capacités suivantes :

- **Ordinateur (HP) :**
 - Processeur : Intel Core i3-1115G4
 - Mémoire : 8 Go
 - Disque dur : SSD 512 Go
 - Carte graphique : Intel UHD Graphics
 - Système d'exploitation : Windows 11

4.2.2 Environnement logiciel

Environnement de développement

- **VS Code :** Visual Studio Code est un éditeur de code source et un environnement de développement intégré (IDE) de Microsoft. Il est open-source et cross-platform, fonctionnant sur Windows, Linux et Mac. Il a été conçu pour les développeurs web, mais prend en charge de nombreux autres langages de programmation. Il est également extensible à l'aide d'une grande variété d'extensions développées par la communauté [6].



FIGURE 4.1 – Logo de VS Code.

Frameworks et outils de développement

- **React** : React est une bibliothèque JavaScript frontale open-source permettant de créer des interfaces utilisateur ou des composants d'interface utilisateur [7].

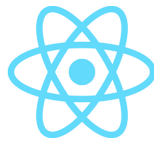


FIGURE 4.2 – Logo de React.

- **Django** : Django est un framework web open source en Python. Il a pour but de rendre le développement d'applications web simple et basé sur la réutilisation de code. [8].



FIGURE 4.3 – Logo de Django.

- **MongoDB** : MongoDB est une base de données NoSQL orientée documents. Elle permet de stocker des données sous forme de documents JSON, offrant une grande flexibilité et des performances élevées. MongoDB Compass est l'interface graphique qui permet d'interroger, d'optimiser et d'analyser les données [9].



FIGURE 4.4 – Logo de MongoDB.

4.3 Principales interfaces graphiques

Cette section présente les captures d'écran des interfaces les plus importantes de l'application.

4.3.1 Interface Tableau de bord Etudiant

L'interface du tableau de bord étudiant affiche le taux de présence global, la liste des matières suivies avec le détail des absences et retards, ainsi que les alertes critiques lorsque l'étudiant dépasse le seuil autorisé.

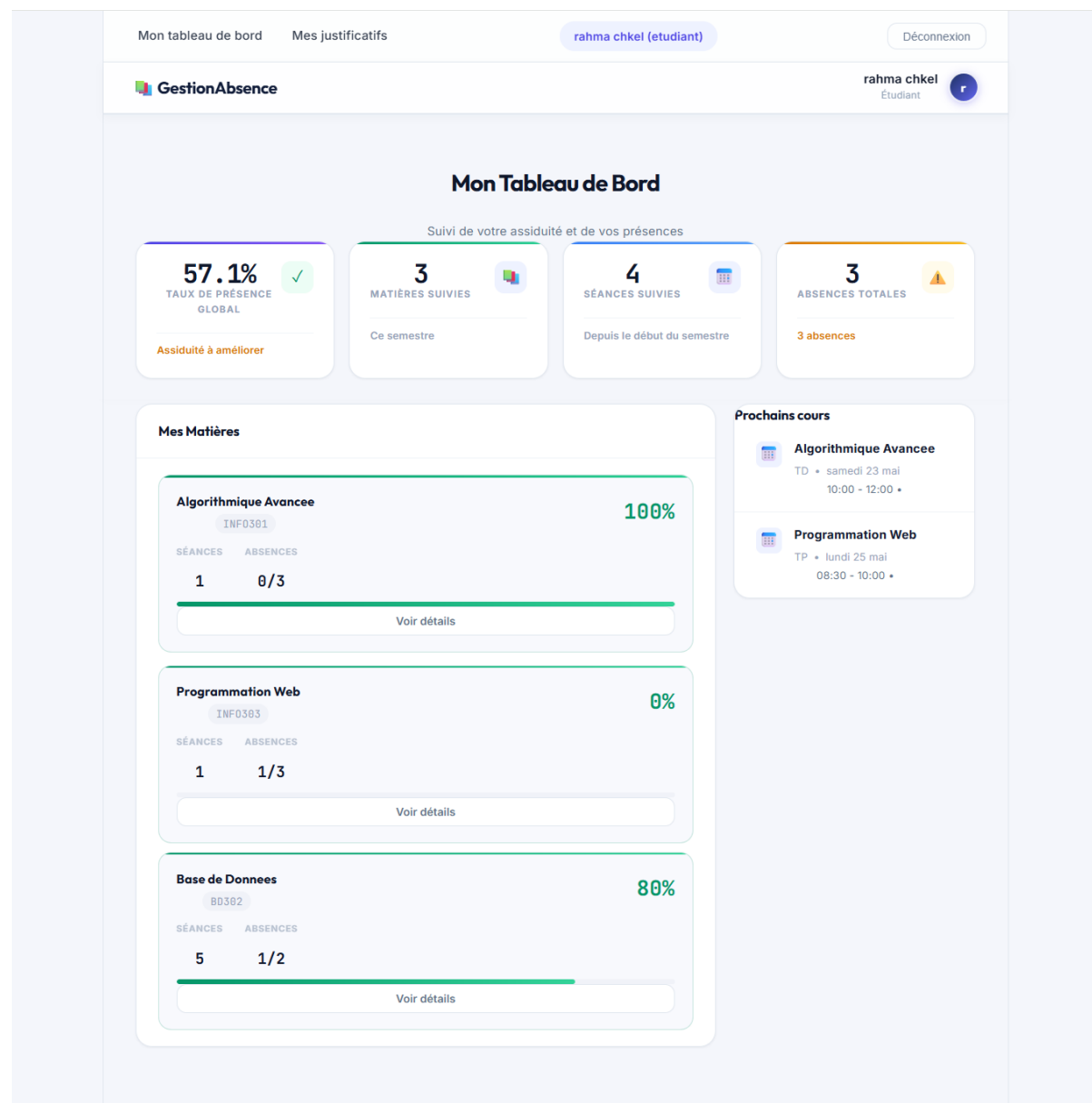


FIGURE 4.5 – Interface Tableau de bord étudiant.

4.3.2 Interface Justificatifs étudiant

L'interface des justificatifs permet à l'étudiant de déposer un document (certificat médical, etc.) pour une absence donnée, de consulter l'état de sa demande (en attente, validé, refusé) et de suivre l'historique de ses justifications.

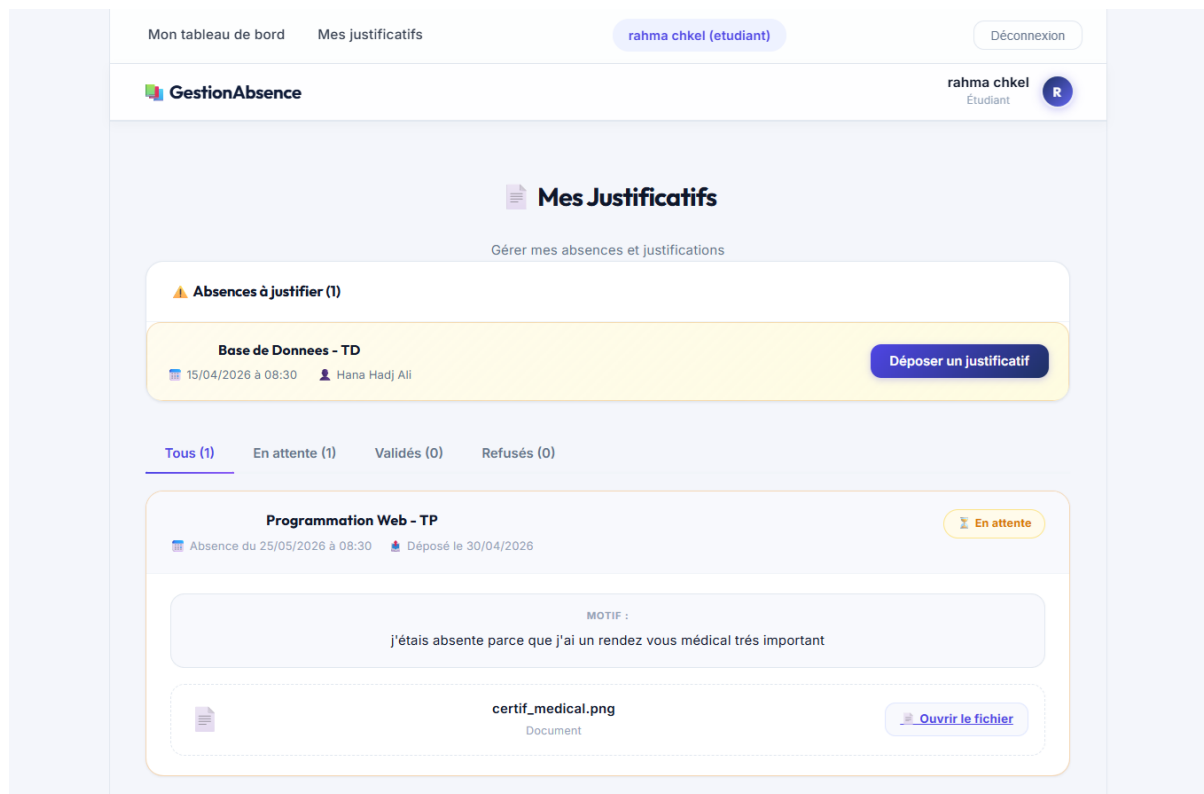


FIGURE 4.6 – Interface Justificatifs étudiant.

4.3.3 Tableau de bord de l'enseignant

L'interface suivante présente le tableau de bord de l'enseignant, offrant un aperçu des groupes assignés, du taux de présence global et des alertes actives.

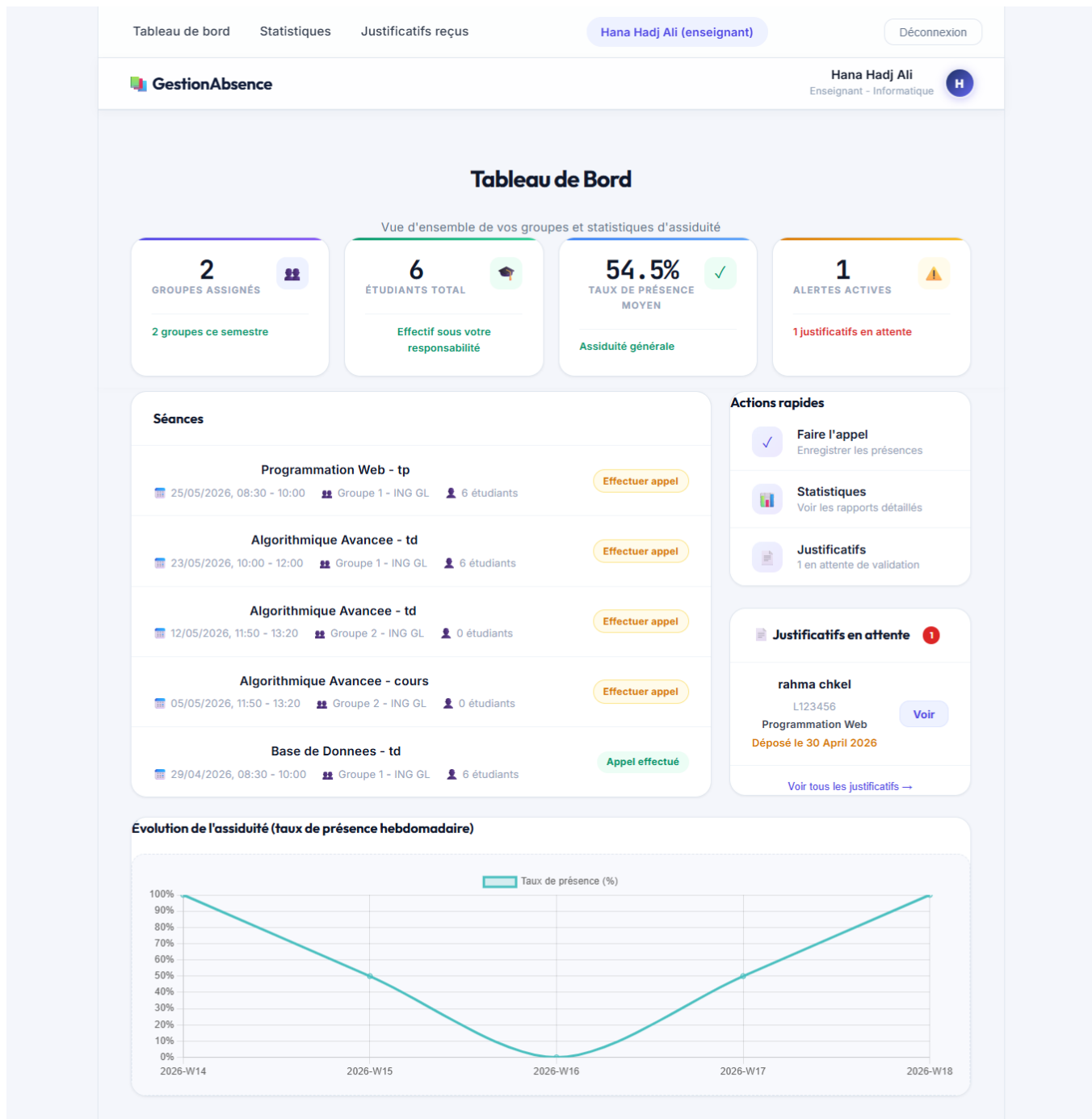


FIGURE 4.7 – Tableau de bord de l'enseignant.

4.3.4 Interface "Faire l'appel"

L'interface de prise d'appel permet à l'enseignant d'enregistrer les présences des étudiants. Pour chaque étudiant, trois statuts sont disponibles : Présent, Absent ou Retard. Un résumé des statistiques (présents, absents, retards) est affiché en temps réel.

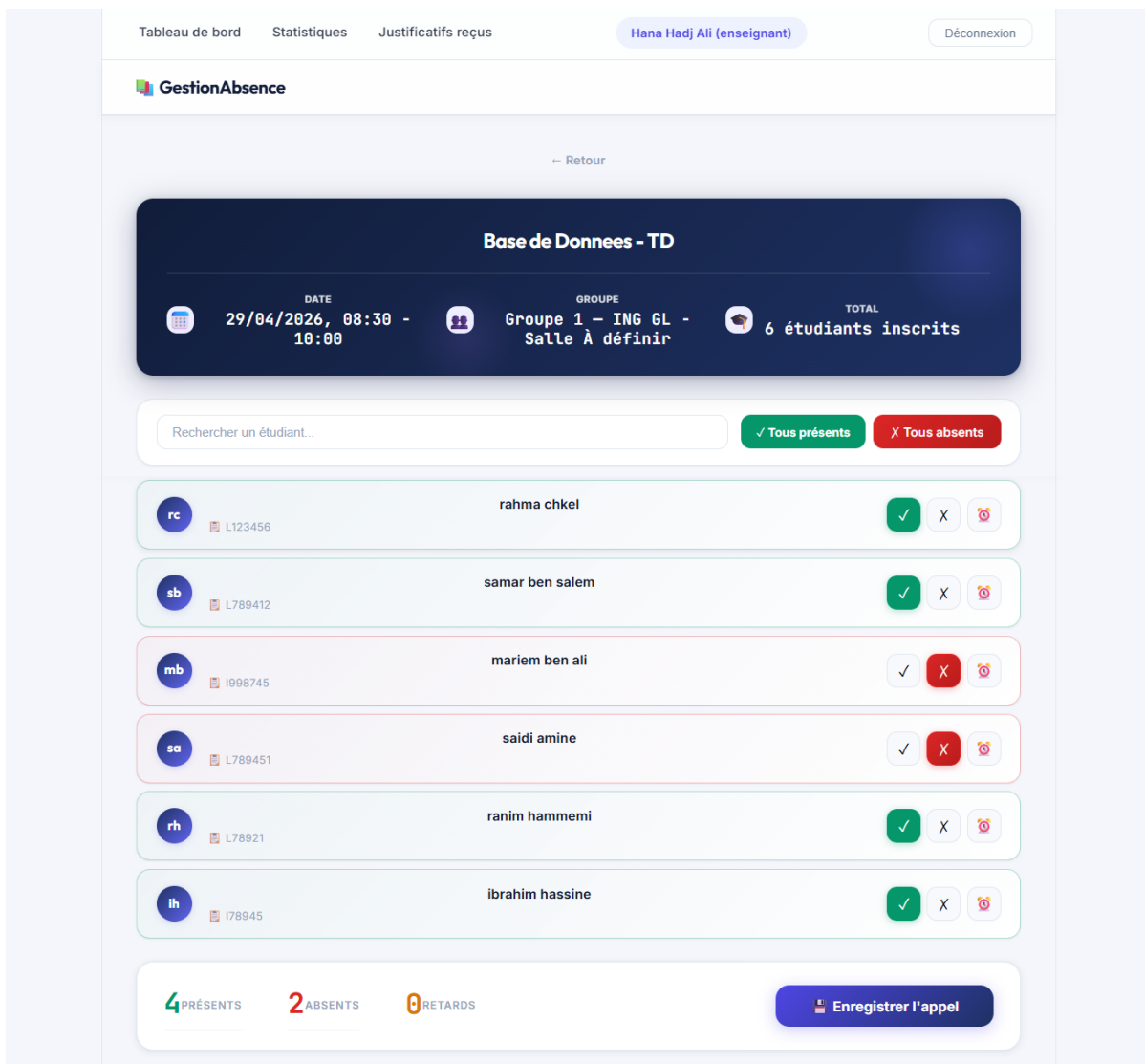


FIGURE 4.8 – Interface de prise d'appel.

4.3.5 Interface de gestion des justificatifs

Cette interface permet à l'enseignant de consulter et valider les justificatifs déposés par les étudiants. Il peut visualiser le document, ajouter un commentaire et accepter ou refuser la justification.

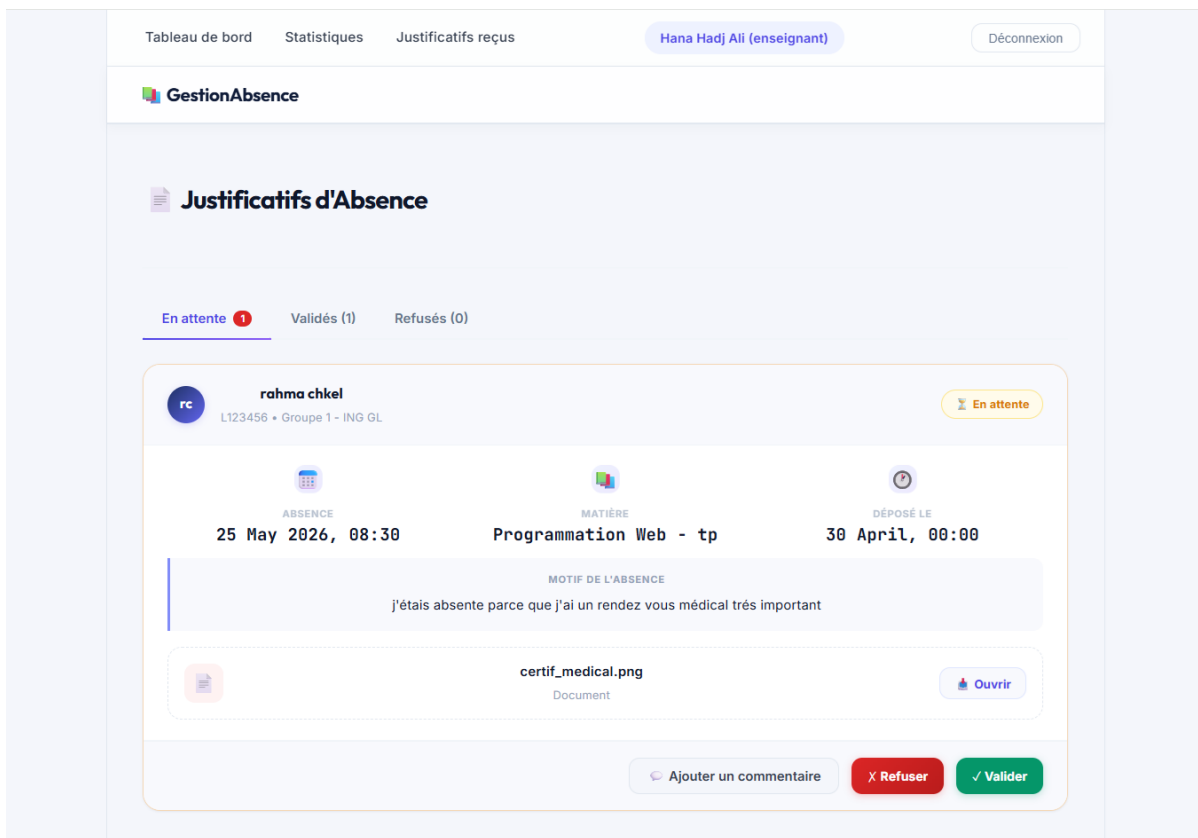


FIGURE 4.9 – Interface de gestion des justificatifs.

4.3.6 Tableau de bord de l'agent administratif

L'interface du tableau de bord administratif présente une vue synthétique de l'absentéisme dans l'établissement : indicateurs globaux (effectifs, taux de présence, alertes, éliminations), statistiques par département, alertes critiques signalant les étudiants en dépassement de seuil, et des actions rapides pour générer des rapports ou gérer les éliminations.

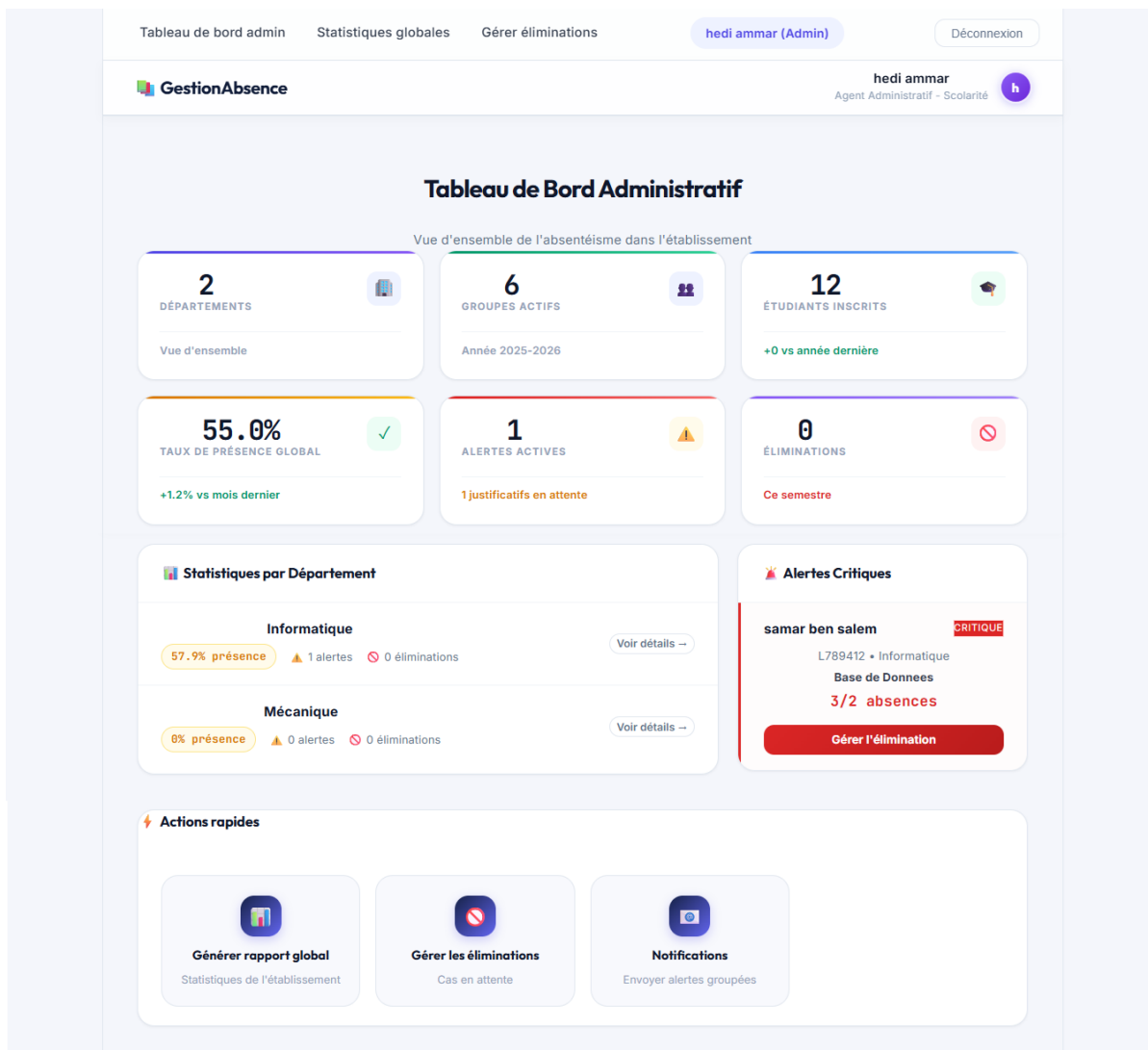


FIGURE 4.10 – Tableau de bord de l'agent administratif.

4.3.7 Interface des statistiques globales

L'interface des statistiques globales offre une analyse complète de l'assiduité dans l'établissement : évolution temporelle du taux de présence, volumes mensuels d'absences et d'éliminations, comparaison par filière (radar et classement), répartition des absences par type de séance (cours/TD/TP), état des justificatifs, et suivi hebdomadaire détaillé des filières ainsi que la liste des étudiants à risque d'élimination

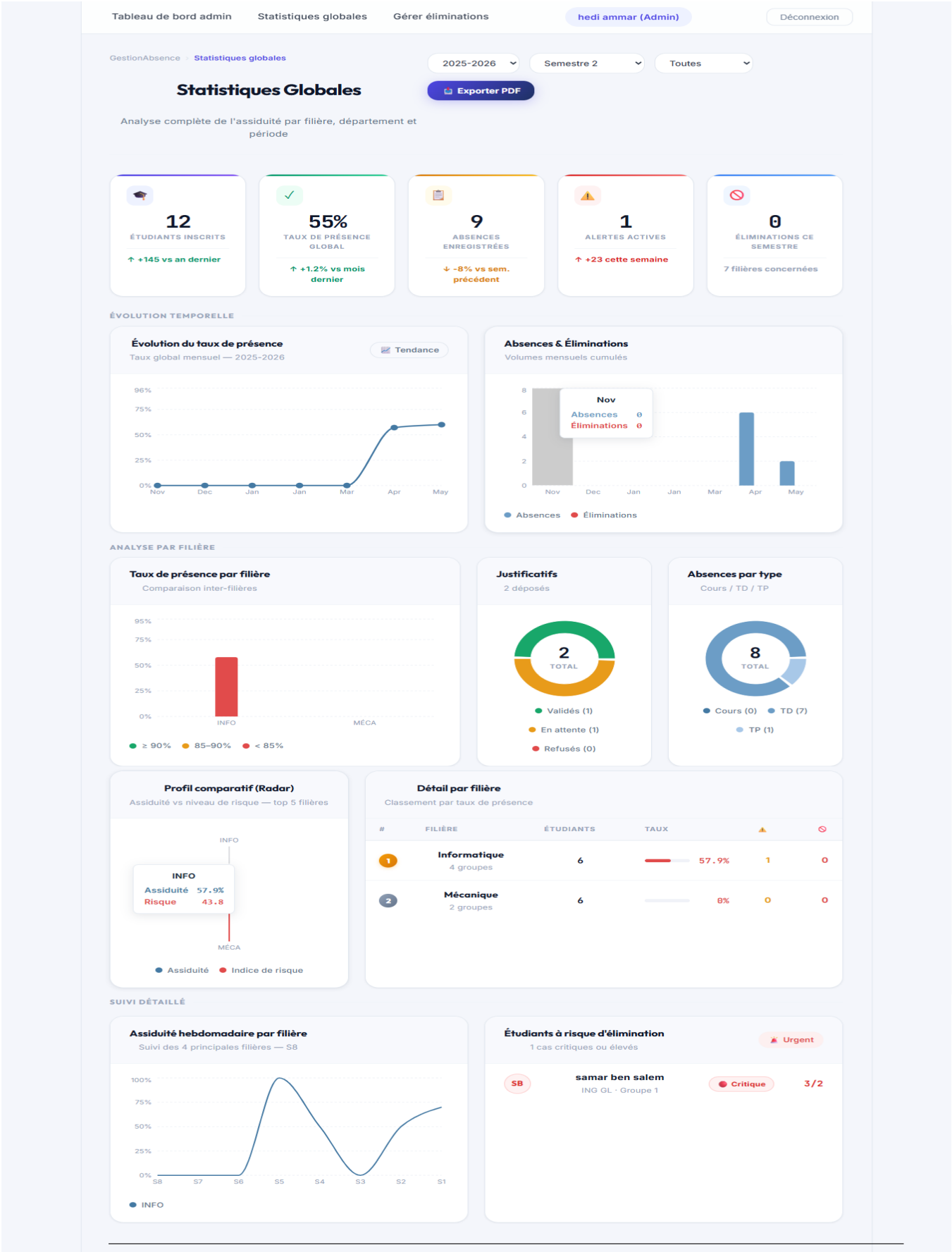


FIGURE 4.11 – Interface des statistiques globales.

4.3.8 Interface de gestion des éliminations

L'interface de gestion des éliminations permet à l'administrateur de suivre les cas d'absentéisme critique, de consulter le détail des absences par étudiant et matière, et de confirmer ou d'annuler les éliminations.

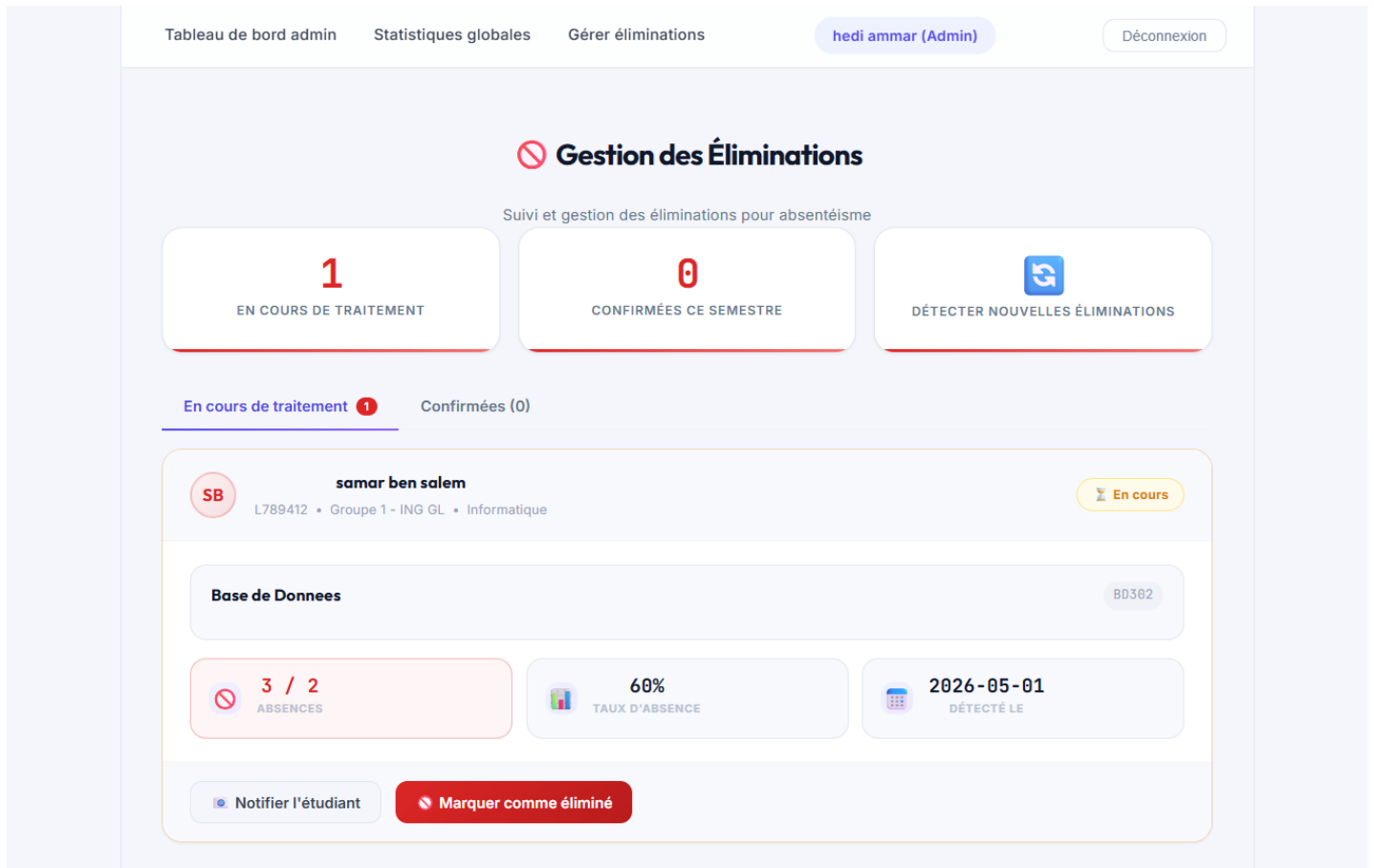


FIGURE 4.12 – Interface de gestion des éliminations.

4.4 Conclusion

Dans ce chapitre, on a présenté la réalisation qu'on a effectué, l'environnement de développement, les outils et technologies utilisés dans le projet ainsi qu'une description détaillée des différentes interfaces utilisateur de l'application.

Conclusion générale et perspectives

Au terme de ce mini projet académique, nous avons conçu et réalisé une application web complète pour la gestion de l'absentéisme au sein de notre institut. Ce travail nous a permis de mettre en pratique nos compétences en développement full-stack, avec React pour l'interface utilisateur, Django (MongoEngine) pour l'API REST, et une authentification JWT. L'application répond aux besoins exprimés par les étudiants, les enseignants et l'agent administratif : suivi personnalisé des présences, dépôt et validation de justificatifs, prise d'appel intuitive, alertes automatiques en cas de dépassement du seuil d'absence, et tableaux de bord analytiques. Cette expérience a renforcé notre maîtrise des architectures modernes et notre capacité à mener un projet fonctionnel de A à Z.

En perspective, plusieurs axes d'amélioration sont envisagés : la réalisation d'une version mobile (application native ou PWA) pour faciliter la prise d'appel et la consultation nomade, le développement d'un modèle de prédiction des tendances d'absence chez les étudiants (à partir des données historiques) afin d'anticiper les risques d'élimination, et enfin le déploiement réel de la solution au sein de l'institut pour une validation en conditions réelles et une intégration pérenne dans le système d'information pédagogique.

Webographie

- [1] “Uml (informatique).” [https://fr.wikipedia.org/wiki/UML_\(informatique\)](https://fr.wikipedia.org/wiki/UML_(informatique)). [Consulté le 15 Avril 2026].
- [2] “Architecture logicielle.” https://fr.wikipedia.org/wiki/Architecture_logicielle. [Consulté le 2 Mars 2026].
- [3] “Architecture trois tiers.” <https://www.ibm.com/fr-fr/think/topics/three-tier-architecture>. [Consulté le 3 Mars 2026].
- [4] “Diagramme de classes.” https://fr.wikipedia.org/wiki/Diagramme_de_classes. [Consulté le 14 Mars 2026].
- [5] “Diagrammes de séquences.” https://fr.wikipedia.org/wiki/Diagramme_de_s%C3%A9quence. [Consulté le 22 Mars 2024].
- [6] “Visual studio code.” <https://bility.fr/definition-visual-studio-code/>. [Consulté le 12 Avril 2026].
- [7] “React.” <https://www.50a.fr/0/react>. [Consulté le 17 Avril 2026].
- [8] “Django.” [https://fr.wikipedia.org/wiki/Django_\(framework\)](https://fr.wikipedia.org/wiki/Django_(framework)). [Consulté le 28 Avril 2026].
- [9] “Mongodb compass.” <https://www.mongodb.com/fr-fr/products/tools/compass>. [Consulté le 21 Avril 2026].